

Núcleo 1 — Text Mining Aplicado à Adaptação Curricular

Aluna: Maria Clara Bragança

Disciplina: Data Mining

Metodologia: CRISP-DM

Fase 1 — Business Understanding

1. Contexto e Problema de Negócio

O CogniKids é uma plataforma educacional voltada para crianças neurodivergentes, ou seja crianças com TEA (Transtorno do Espectro Autista), TDAH ou Dislexia. O problema central é que um professor produz uma única atividade textual para toda a turma, mas cada criança neurodivergente processa esse texto de forma diferente. Uma criança com dislexia se beneficia de frases curtas e simples, enquanto uma criança com TEA precisa de instruções em passo a passo concreto e sem ambiguidade. Reescrever manualmente a mesma atividade em varios formatos diferentes para cada professor e cada turma nao é uma pratica sustentavel.

2. Objetivo de Negócio

Investigar se é possível extrair automaticamente tres propriedades de um texto pedagogico escrito por um professor antes de adapta-lo para o perfil de cada criança:

- **Complexidade textual:** o texto esta acima do nivel de leitura da criança?
- **Intenção pedagogica (dominio cognitivo, Taxonomia de Bloom):** o professor quer que o aluno memorize um fato, analise, compare ou crie? Essa distinção é importante porque uma adaptação que simplifica demais pode trocar, sem querer, um objetivo de analise por um de memorizacao, ou seja reduziria o valor pedagogico da atividade.
- **Demanda motora (dominio psicomotor, Taxonomia de Simpson):** a atividade pede so raciocinio ou tambem exige execucao fisica, como montar, recortar ou manusear um objeto? Isso importa porque parte do publico do CogniKids tem dificuldade de coordenacao motora associada ao TEA e ao TDAH, e uma atividade com demanda motora alta pode precisar de um tipo de adaptacao diferente de uma atividade so cognitiva.

Essas tres propriedades cobrem, junto com o dominio afetivo ja medido pela pulseira IoT do sistema (BPM e GSR, indicando estado emocional), os tres dominios classicos da aprendizagem descritos na literatura (Bloom 1956, Simpson 1972, Krathwohl 1964). A arquitetura de adaptar formato sem mudar conteudo tambem é fundamentada no

framework Universal Design for Learning (UDL 3.0, CAST 2024), detalhado na conclusao deste notebook.

3. Objetivo da Mineração de Dados

Analisar datasets publicos disponíveis para verificar se métricas calculaveis de complexidade textual e classificacao de intencao pedagogica conseguem caracterizar o texto de entrada de forma confiavel, sem precisar de um modelo pesado ou de um servico externo. Estender a mesma abordagem para o dominio psicomotor, onde nao existe dataset publico disponivel em nenhum idioma.

4. Perguntas Estratégicas

- Existe correlação suficiente entre uma métrica calculavel de complexidade e a avaliacao humana de facilidade de leitura?
 - É possivel classificar a intencao pedagogica de um texto escrito em portugues sem depender de um dataset em outro idioma?
 - Um sistema baseado em regras de verbos resolve o problema melhor do que um modelo treinado em ingles aplicado a texto em portugues?
 - É possivel identificar, pelo mesmo tipo de sistema baseado em regras, se uma atividade tem demanda motora (dominio psicomotor), mesmo sem nenhum dataset disponivel para esse dominio?
-

Fase 2 — Data Understanding

Importação de bibliotecas e carregamento dos datasets

```
In [1]: # Eu importo o pandas para trabalhar com os datasets em formato de tabela
import pandas as pd

# Eu importo o textstat para calcular metricas de legibilidade de texto em
# ingles
import textstat

# Eu importo o train_test_split para separar treino e teste sem data leakage
from sklearn.model_selection import train_test_split

# Eu importo o TfidfVectorizer para transformar texto em numeros que o modelo
# consegue processar
from sklearn.feature_extraction.text import TfidfVectorizer

# Eu importo a Regressao Logistica como classificador baseline para o texto em
# ingles
from sklearn.linear_model import LogisticRegression

# Eu importo as funcoes de avaliacao para medir o desempenho do classificador
from sklearn.metrics import classification_report, accuracy_score
```

```
# Eu defino o caminho base onde estao os datasets utilizados nessa analise
DATASETS = "../..../datasets/raw"
```

2.1 Dataset CLEAR Corpus — complexidade textual

O CLEAR Corpus é um dataset publico com 4.726 trechos de texto em ingles, cada um avaliado por leitores humanos quanto a facilidade de leitura. As colunas que me interessam sao:

- **Excerpt:** o trecho de texto
- **BT Easiness:** nota de facilidade de leitura atribuida por avaliacao humana, onde valores maiores indicam texto mais facil
- **Flesch-Kincaid-Grade-Level:** metrica calculavel que estima a serie escolar necessaria para compreender o texto, ou seja nao depende de avaliacao humana e valores maiores indicam maior dificuldade

A pergunta que eu quero responder aqui é: existe correlacao suficiente entre a metrica calculavel e a avaliacao humana para eu poder usar a primeira como representante da segunda, sem precisar treinar um modelo de regressao dedicado? Pois se a correlacao for boa o suficiente eu consigo calcular a complexidade de qualquer texto novo sem depender de avaliadores humanos.

Fase 3 — Preparação dos Dados (CLEAR Corpus)

Eu seleciono apenas as tres colunas relevantes e removo as linhas com valores ausentes, pois um registro sem nota de legibilidade nao é utilizavel na comparacao. Eu tambem recalculo o Flesch-Kincaid usando a biblioteca textstat em vez de usar o valor ja calculado do dataset, pois preciso reproduzir exatamente o mesmo procedimento que vai ser aplicado a um texto novo que ainda nao tem esse calculo pronto.

```
In [2]: # Eu carrego o dataset CLEAR Corpus em formato CSV
clear = pd.read_csv(f"{DATASETS}/commonlit_clear_corpus/CLEAR.csv")

# Eu seleciono apenas as colunas que preciso e removo linhas com valores
ausentes
clear = clear[["Excerpt", "Flesch-Kincaid-Grade-Level", "BT
Easiness"]].dropna()

# Eu recalculo o Flesch-Kincaid com o textstat porque é esse calculo que vai
rodar em texto novo
# e nao o valor pre-calculado que ja vem no dataset
clear["fk_calculado"] = clear["Excerpt"].apply(textstat.flesch_kincaid_grade)

# Eu calculo a correlacao entre o Flesch-Kincaid calculado e a nota humana de
facilidade
correlacao = clear["fk_calculado"].corr(clear["BT Easiness"])

# Eu mostro o valor da correlacao para interpretar se a metrica é uma boa
representante
print(f"Correlação Flesch-Kincaid (calculado) vs. nota humana de facilidade:
{correlacao:.3f}")
```

```
# Eu mostro as primeiras linhas do dataset preparado para conferir a estrutura
clear.head()
```

Correlação Flesch-Kincaid (calculado) vs. nota humana de facilidade: -0.500

Out[2]:

	Excerpt	Flesch-Kincaid-Grade-Level	BT Easiness	fk_calculado
0	When the young people returned to the ballroom...	5.95	-0.340259	6.247984
1	All through dinner time, Mrs. Fayre was somewh...	4.86	-0.315372	5.246851
2	As Roger had predicted, the snow departed as q...	6.03	-0.580118	5.798976
3	Mr. Grimes was to come up next morning to Sir ...	20.51	-1.785965	19.996981
4	And outside before the palace a great garden w...	12.06	-1.054013	11.592244

Interpretação

A correlacao obtida foi de **-0.5**, ou seja uma correlacao negativa de magnitude moderada na direcao que eu esperava: quanto maior o Flesch-Kincaid (texto mais dificil), menor a nota humana de facilidade. Isso faz sentido porque as duas metricas medem a mesma coisa em direcoes opostas.

O valor de -0.5 nao é perfeito, o que indica que a metrica calculavel nao consegue reproduzir exatamente o julgamento humano, pois avaliadores humanos consideram fatores que nenhuma formula captura, como contexto cultural e familiaridade com o tema. Mas a correlacao é forte o suficiente para justificar o uso do Flesch-Kincaid como representante da complexidade textual, ou seja eu consigo caracterizar a dificuldade de um texto sem precisar de avaliadores humanos nem de um modelo de regressao treinado.

Isso interfere diretamente no objetivo da analise: se a correlacao fosse proxima de zero eu teria que treinar um modelo de regressao dedicado, o que exigiria dados rotulados e aumentaria a complexidade do sistema. Com -0.5 a formula calculavel ja resolve o problema de forma simples.

Fase 4 e 5 — Modelagem e Avaliação: funcao de complexidade textual

Agora eu crio a funcao que vai calcular a complexidade de qualquer texto novo usando as metricas do textstat que eu validei contra o CLEAR Corpus. Eu uso um texto de exemplo sobre fotossintese para testar o funcionamento antes de avancar.

```
In [3]: # Eu defino a funcao que calcula a complexidade textual de um texto em ingles
def avaliar_complexidade(texto: str) -> dict:
    # Eu calculo o nivel de serie escolar estimado pelo Flesch-Kincaid
```

```

fk_grade = round(textstat.flesch_kincaid_grade(texto), 2)

# Eu calculo o indice de facilidade de leitura Flesch (escala 0 a 100,
maior = mais facil)
fk_ease = round(textstat.flesch_reading_ease(texto), 2)

# Eu conto o numero de palavras do texto
n_palavras = textstat.lexicon_count(texto)

# Eu retorno um dicionario com as tres metricas calculadas
return {
    "flesch_kincaid_grade": fk_grade,
    "flesch_reading_ease": fk_ease,
    "contagem_palavras": n_palavras,
}

# Eu defino o texto de exemplo que vou usar ao longo de todo o notebook
texto_exemplo = (
    "A fotossíntese é o processo pelo qual as plantas convertem luz solar, "
    "água e dióxido de carbono em glicose e oxigênio, utilizando a clorofila "
    "presente em suas células para capturar a energia luminosa."
)

# Eu testo a funcao com o texto de exemplo para ver os valores de saida
avaliar_complexidade(texto_exemplo)

```

```

Out[3]: {'flesch_kincaid_grade': 18.73,
        'flesch_reading_ease': 19.52,
        'contagem_palavras': 33}

```

Interpretação

A funcao retorna tres metricas para qualquer texto de entrada. O texto de exemplo sobre fotossíntese vai gerar valores que refletem a dificuldade real do texto, ou seja eu consigo verificar se os numeros fazem sentido: um texto científico com termos técnicos como 'dióxido de carbono' e 'clorofila' deveria ter um Flesch-Kincaid alto (serie avancada) e um indice de facilidade baixo.

Essa funcao é o que vai ser chamada para caracterizar o texto do professor antes de qualquer adaptacao, ou seja ela é a primeira etapa do pipeline de analise. Na proxima secao eu vou investigar a segunda propriedade: a intencao pedagogica.

2.2 Dataset Bloom's Taxonomy — intenção pedagógica

A Taxonomia de Bloom organiza os objetivos de aprendizagem em 6 niveis cognitivos em ordem crescente de complexidade. Eu preciso entender essa estrutura porque é ela que define o que o professor quer que o aluno faça com o conteudo:

Nível	Nome	O que o aluno faz	Verbos típicos
BT1	Lembrar	Recordar um fato	listar, definir
BT2	Entender	Explicar com as proprias palavras	explicar, resumir

Nível	Nome	O que o aluno faz	Verbos típicos
BT3	Aplicar	Usar o conhecimento em situação nova	resolver, calcular
BT4	Analisar	Decompor e relacionar partes	comparar, categorizar
BT5	Avaliar	Julgar segundo critérios	criticar, justificar
BT6	Criar	Produzir algo original	elaborar, formular

O dataset que eu vou usar tem 8.767 perguntas de atividades escolares em inglês, cada uma rotulada com o nível BT correspondente. Isso é um problema de classificação supervisionada, ou seja, dado um texto de entrada eu preciso prever uma categoria entre as 6 possíveis a partir de exemplos já rotulados.

Fase 3 e 4 — Preparação e Modelagem: classificador de Bloom em inglês

Eu separei 80% dos dados para treino e 20% para teste. É importante fazer essa separação antes de qualquer transformação para evitar data leakage, que é quando o modelo vê informações que não deveria e aprende a 'colar' em vez de aprender o padrão real. O parâmetro **stratify** garante que a proporção de cada nível BT seja igual nos dois conjuntos, ou seja, a classe minoritária não vai ficar sub-representada no teste por acaso.

Para o classificador eu uso o TF-IDF que transforma cada texto em um vetor de pesos por palavra. Ele dá peso menor para palavras muito comuns em todo o corpus (como artigos e preposições) e peso maior para palavras raras e específicas, o que indica que os verbos característicos de cada nível de Bloom vão ter peso alto porque aparecem pouco no geral mas muito em textos daquele nível específico.

Seleção do algoritmo: por que eu não posso simplesmente escolher um

Antes de treinar o classificador final, eu preciso responder uma pergunta que eu não podia pular: qual algoritmo é o melhor para esse problema? Eu não posso simplesmente usar Regressão Logística porque é o mais comum em tutoriais — isso não é uma justificativa válida num trabalho acadêmico. Eu preciso comparar candidatos com o mesmo critério e escolher com base em número, não em costume.

Eu separei 5 algoritmos diferentes para comparar, todos usando o mesmo TF-IDF (para a comparação ser justa, a única coisa que muda é o classificador):

- **Regressão Logística:** aprende um peso linear por palavra para cada classe
- **Naive Bayes Multinomial:** muito usado em classificação de texto, assume que as palavras são independentes entre si
- **SVM Linear (LinearSVC):** procura a fronteira que separa as classes com a maior margem possível
- **Random Forest:** combina várias árvores de decisão treinadas em subconjuntos diferentes dos dados

- **KNN:** classifica um texto novo olhando para os k vizinhos mais parecidos no conjunto de treino

Para cada um eu uso **validacao cruzada com 5 folds** (5-fold cross-validation) no conjunto de treino, ou seja eu divido o treino em 5 partes, treino em 4 e valido na parte restante, repetindo 5 vezes com uma parte diferente de cada vez. Isso me da uma estimativa de desempenho mais confiavel do que um unico split, porque reduz a chance de eu escolher um modelo que so foi bom por sorte numa divisao especifica dos dados.

```
In [4]: # Eu importo os classificadores adicionais que vou comparar com a Regressao
Logistica
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import f1_score
import time

# Eu carrego o dataset de Bloom's Taxonomy com as perguntas rotuladas por nivel
cognitivo
bloom = pd.read_csv(f"{DATASETS}/blooms_taxonomy/blooms_taxonomy_dataset.csv")

# Eu mostro a distribuicao dos niveis para ver se a base é balanceada ou nao
print(bloom["Category"].value_counts())

# Eu separo X (texto da pergunta) e y (nivel de Bloom) e faço o split
estratificado
X_train, X_test, y_train, y_test = train_test_split(
    bloom["Questions"], bloom["Category"],
    test_size=0.2,          # Eu reservo 20% dos dados para testar o
                           # modelo
    random_state=42,        # Garante que o resultado seja reproduzível
    stratify=bloom["Category"] # Eu mantenho a proporcao de cada nivel nos
                             # dois conjuntos
)

# Eu crio o vetorizador TF-IDF para transformar texto em numeros
vetorizador = TfidfVectorizer(
    max_features=5000,      # Eu limito a 5000 palavras mais relevantes
    ngram_range=(1, 2),    # Eu incluo pares de palavras alem de palavras
                           # individuais
    stop_words="english"    # Eu removo palavras muito comuns em ingles que
                           # nao ajudam
)

# Eu aplico o TF-IDF no treino – o vetorizador aprende o vocabulario aqui
X_train_vec = vetorizador.fit_transform(X_train)

# Eu aplico o mesmo TF-IDF no teste – sem re-aprender, so transformar
X_test_vec = vetorizador.transform(X_test)

# Eu defino os candidatos que vou comparar, todos usando o mesmo TF-IDF acima
candidatos = {
    "LogisticRegression": LogisticRegression(max_iter=1000),
    "MultinomialNB":      MultinomialNB(),
    "LinearSVC":           LinearSVC(max_iter=2000),
```

```

    "RandomForest": RandomForestClassifier(n_estimators=200,
random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
}

# Eu configuro a validacao cruzada estratificada com 5 folds
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Eu crio uma lista para guardar os resultados de cada candidato
resultados = []

# Eu treino e avalio cada candidato com o mesmo criterio
for nome, modelo in candidatos.items():
    # Eu rodo a validacao cruzada no conjunto de treino para estimar o
    desempenho de forma robusta
    # (n_jobs=1 aqui de proposito: em Windows o paralelismo do joblib gera
    ruido de log sem ganho real nesse tamanho de dado)
    cv_scores = cross_val_score(modelo, X_train_vec, y_train, cv=cv,
scoring="accuracy", n_jobs=1)

    # Eu meço o tempo de treino porque isso tambem importa na escolha do modelo
    inicio = time.time()
    modelo.fit(X_train_vec, y_train)
    tempo_treino = time.time() - inicio

    # Eu avalio no conjunto de teste, que o modelo nunca viu
    preds = modelo.predict(X_test_vec)
    acc_teste = accuracy_score(y_test, preds)
    f1_teste = f1_score(y_test, preds, average="macro")

    # Eu guardo os resultados para montar a tabela comparativa
    resultados.append({
        "modelo": nome,
        "cv_acc_media": round(cv_scores.mean(), 3),
        "cv_acc_desvio": round(cv_scores.std(), 3),
        "teste_acc": round(acc_teste, 3),
        "teste_f1_macro": round(f1_teste, 3),
        "tempo_treino_s": round(tempo_treino, 2),
    })

# Eu monto a tabela comparativa final ordenada pela acuracia de validacao
cruzada
comparacao = pd.DataFrame(resultados).sort_values("cv_acc_media",
ascending=False)
comparacao

```

Category

BT1	2582
BT2	1801
BT3	1508
BT4	1293
BT6	800
BT5	783

Name: count, dtype: int64

Out[4]:

	modelo	cv_acc_media	cv_acc_desvio	teste_acc	teste_f1_macro	tempo_treii
3	RandomForest	0.754	0.005	0.751	0.778	2
2	LinearSVC	0.726	0.009	0.738	0.767	
0	LogisticRegression	0.720	0.008	0.727	0.752	
1	MultinomialNB	0.675	0.011	0.682	0.691	
4	KNN	0.389	0.008	0.404	0.338	

Interpretação

O **Random Forest venceu em todas as métricas**: maior acurácia de validação cruzada (0.754), maior acurácia no teste (0.751) e maior F1-macro (0.778), além do menor desvio padrão entre os folds (0.005), ou seja é o modelo mais consistente entre os candidatos. O custo é o tempo de treino, cerca de 11 segundos contra menos de 1 segundo dos outros — mas para um dataset de 8.767 exemplos esse custo é irrelevante em termos absolutos, então não existe motivo para eu abrir mão do melhor desempenho por causa disso.

O **LinearSVC** ficou muito próximo do Random Forest (0.738 de acurácia, 0.767 de F1) com um tempo de treino muito menor, ou seja seria a escolha certa se o tempo de treino fosse um fator limitante — o que não é o caso aqui.

A **Regressão Logística**, que eu tinha usado sem justificar no notebook anterior, ficou em terceiro lugar (0.727 de acurácia). Ela não era o melhor modelo, e eu só teria descoberto isso se eu tivesse comparado — o que reforça que escolher um algoritmo sem comparar candidatos não é uma prática válida.

O **KNN teve o pior desempenho** (0.404 de acurácia), bem abaixo dos outros. Isso tem uma explicação técnica: o TF-IDF gera vetores com até 5.000 dimensões, e o KNN mede distância entre pontos nesse espaço. Em espaços de alta dimensionalidade a noção de 'vizinho próximo' perde significado, porque quase todos os pontos ficam a distâncias parecidas entre si — um fenômeno conhecido como **maldição da dimensionalidade**. O KNN funciona bem em poucas dimensões, mas não é adequado para dados de texto vetorizados dessa forma.

Decisão: eu escolho o Random Forest para o classificador final desta seção, com base na acurácia de validação cruzada, na acurácia de teste e no F1-macro, todos superiores aos demais candidatos, e um custo de treino que continua desprezível para o tamanho deste dataset.

Treino do modelo final

Com o algoritmo escolhido e justificado, eu treino o Random Forest como o classificador oficial desta seção, usando o mesmo split e o mesmo TF-IDF da comparação.

```
In [5]: # Eu já tenho X_train_vec, X_test_vec, y_train, y_test e o vetorizador prontos
da comparacao acima,
```

```
# entao eu so preciso treinar o modelo vencedor (Random Forest) como o
classificador oficial desta secao

# Eu crio o classificador final com os mesmos hiperparametros usados na
comparacao
classificador_bloom = RandomForestClassifier(n_estimators=200, random_state=42)

# Eu treino o classificador com o conjunto de treino
classificador_bloom.fit(X_train_vec, y_train)

# Eu gero as predicoes no conjunto de teste
preds = classificador_bloom.predict(X_test_vec)

# Eu mostro a acuracia geral e o relatorio detalhado por classe
print(f"Acurácia: {accuracy_score(y_test, preds):.3f}")
print(classification_report(y_test, preds))
```

Acurácia: 0.751

	precision	recall	f1-score	support
BT1	0.64	0.84	0.73	516
BT2	0.75	0.59	0.66	360
BT3	0.77	0.75	0.76	302
BT4	0.86	0.64	0.73	259
BT5	0.92	0.87	0.90	157
BT6	0.89	0.89	0.89	160
accuracy			0.75	1754
macro avg	0.81	0.76	0.78	1754
weighted avg	0.77	0.75	0.75	1754

Fase 5 — Avaliação: métricas do classificador final (Random Forest) em inglês

Antes de interpretar os numeros eu preciso entender o que cada metrica significa:

- **Acuracia:** proporcao de classificacoes corretas sobre o total de exemplos de teste
- **Precision:** de todos os textos que o modelo disse que eram de um nivel, quantos realmente eram
- **Recall:** de todos os textos que realmente pertencem a um nivel, quantos o modelo identificou corretamente
- **F1-score:** media harmonica entre Precision e Recall, ou seja penaliza o modelo que é muito bom em uma metrica mas pessimo na outra

Interpretação

A acuracia de **75.1%** no Random Forest indica que o modelo acertou aproximadamente 75 de cada 100 classificacoes no conjunto de teste, um resultado melhor que os 72.7% da Regressao Logistica que eu tinha usado antes de comparar os candidatos. BT5 (avaliar) e BT6 (criar) continuam com os melhores F1-scores (0.90 e 0.89) porque esses niveis usam verbos muito caracteristicos como 'critique' e 'elaborate' que aparecem pouco fora deles. Ja BT2 (entender) continua sendo o nivel com maior taxa de confusao (F1 = 0.66, recall de apenas 0.59) porque fica no meio da escala, ou seja semanticamente

proximo do BT1 e do BT3 ao mesmo tempo — esse padrao se repete independente do algoritmo escolhido, o que indica que a dificuldade esta no proprio dado, nao no modelo.

Esse resultado confirma que a relacao entre vocabulario e nivel cognitivo é aprendivel a partir de dados, e que a comparacao de algoritmos valeu a pena: eu ganhei mais de 2 pontos percentuais de acuracia e quase 3 pontos de F1-macro so por escolher o modelo certo em vez do mais comum. Mas como eu vou mostrar a seguir, esse classificador especifico nao vai para producao pois existe um problema importante que a acuracia nao revela: o modelo foi treinado e testado em ingles, mas o CogniKids recebe textos de professores brasileiros em portugues.

Funcao auxiliar e teste em inglês

Eu crio uma funcao auxiliar para facilitar a classificacao de novos textos e testo com uma frase em ingles para confirmar que o modelo funciona como esperado antes de testar com texto em portugues.

```
In [6]: # Eu crio a funcao auxiliar que recebe um texto e retorna o nivel de Bloom  
previsto  
def classificar_intencao_pedagogica(texto: str) -> str:  
    # Eu transformo o texto em vetor TF-IDF usando o vetorizador ja treinado  
    vetor = vetorizador.transform([texto])  
  
    # Eu retorno a predicao do classificador para esse vetor  
    return classificador_bloom.predict(vetor)[0]  
  
# Eu testo com uma frase em ingles para confirmar que o modelo classifica  
corretamente  
classificar_intencao_pedagogica("Explain why plants need sunlight to survive.")
```

```
Out[6]: 'BT1'
```

Fase 5 — Avaliação: testando o classificador com texto em português

Agora eu testo o mesmo classificador treinado em ingles com o texto de exemplo em portugues que eu defini no inicio do notebook. A pergunta que eu quero responder é: o modelo consegue generalizar para um idioma diferente do que ele foi treinado?

```
In [7]: # Eu testo o mesmo classificador treinado em ingles com texto em portugues  
# pra ver o que acontece quando o modelo encontra um idioma que nao conhece  
classificar_intencao_pedagogica(texto_exemplo)
```

```
Out[7]: 'BT1'
```

Interpretação

O resultado foi **BT1**, ou seja o modelo classificou o texto de fotossintese em portugues como 'lembrar', que é o nivel mais basico da taxonomia. Isso nao faz sentido pois o texto

descreve um processo complexo, não é uma instrução de memorização.

O que aconteceu aqui tem um nome técnico: mudança de domínio. O TF-IDF aprendeu um vocabulário inteiramente em inglês, com palavras como 'explain', 'compare' e 'critique'. Quando recebe um texto em português ele não reconhece nenhuma dessas palavras, ou seja o vetor que representa o texto fica praticamente zerado em todas as dimensões que importam. Sem sinal útil, o modelo cai na classe mais frequente do treino que é o BT1 com 2.582 de 8.767 exemplos — e isso acontece independente do algoritmo escolhido, porque o problema está no vocabulário do vetorizador, não no classificador.

Isso indica que a acurácia de 75.1% medida antes (a melhor entre os 5 candidatos comparados) era válida apenas para textos em inglês do mesmo domínio do treino. Para textos em português essa métrica deixa de ser informativa e o classificador não serve para produção nesse projeto, mesmo sendo o melhor da comparação. Por isso eu preciso de uma abordagem diferente para o português, que é o que eu vou desenvolver a seguir.

Fase 4 (revisitada) — Complexidade textual em português

O mesmo princípio da seção 2.1 (Flesch-Kincaid) é aplicado agora com uma fórmula calibrada especificamente para o português: o Índice Flesch adaptado. A fórmula original em inglês não funciona diretamente para o português porque a contagem de sílabas por palavra é diferente entre os dois idiomas. O português tem em média mais sílabas por palavra, o que indica que aplicar os pesos originais produziria uma subestimativa sistemática da dificuldade de textos em português. A versão adaptada recalibra esses pesos:

$$\text{Índice} = 248.835 - 1.015 \times (\text{palavras} / \text{frases}) - 84.6 \times (\text{sílabas} / \text{palavras})$$

Valores maiores indicam texto mais fácil, na mesma direção da métrica original. A contagem de sílabas é feita pela biblioteca pyphen que implementa as regras de sílabação do português.

```
In [8]: # Eu importo o re para trabalhar com expressões regulares na contagem de frases e palavras
import re

# Eu importo o pyphen para fazer a sílabação correta em português brasileiro
import pyphen

# Eu crio o dicionário de sílabação para o português do Brasil
_dic_pt = pyphen.Pyphen(lang="pt_BR")

# Eu defino a função auxiliar que conta as sílabas de uma palavra em português
def _contar_silabas_pt(palavra: str) -> int:
    # O pyphen insere hifens nos pontos de sílabação, então eu conto os hifens e adiciono 1
    return _dic_pt.inserted(palavra).count("-") + 1

# Eu defino a função principal que calcula o Índice Flesch adaptado para o
```

```

portugues
def avaliar_complexidade_pt(texto: str) -> dict:
    # Eu separo as frases usando pontuacao de fim de frase
    frases = [f for f in re.split(r"[.!?]+", texto) if f.strip()]

    # Eu extraio apenas as palavras (letras com e sem acento)
    palavras = re.findall(r"[A-Za-zÃ-ú]+", texto)

    # Eu verifico se o texto tem frases e palavras para evitar divisao por zero
    if not frases or not palavras:
        return {"indice_flesch_pt": None, "contagem_palavras": 0}

    # Eu conto o total de silabas somando as silabas de cada palavra
    total_silabas = sum(_contar_silabas_pt(p) for p in palavras)

    # Eu calculo a media de palavras por frase (ASL)
    asl = len(palavras) / len(frases)

    # Eu calculo a media de silabas por palavra (ASW)
    asw = total_silabas / len(palavras)

    # Eu aplico a formula Flesch adaptada para o portugues
    indice = 248.835 - 1.015 * asl - 84.6 * asw

    # Eu retorno o indice e as metricas intermediarias para facilitar a
    interpretacao
    return {
        "indice_flesch_pt": round(indice, 2),
        "contagem_palavras": len(palavras),
        "silabas_por_palavra": round(asw, 2),
    }

# Eu testo a funcao com o texto de exemplo para ver os valores de saida
avaliar_complexidade_pt(texto_exemplo)

```

```

Out[8]: {'indice_flesch_pt': 33.32,
        'contagem_palavras': 33,
        'silabas_por_palavra': 2.15}

```

Interpretação

A funcao retorna o indice Flesch adaptado para o texto em portugues. Para o texto de fotossintese que é um texto científico com termos técnicos e frases longas, eu espero um indice baixo, ou seja texto difícil, pois a formula penaliza tanto frases longas quanto muitas silabas por palavra e esse texto tem as duas características.

O campo silabas_por_palavra ajuda a verificar se a contagem de silabas pelo pyphen esta correta para o portugues. A metrica de contagem_palavras tambem é util para identificar textos muito curtos onde o indice pode ser menos confiavel. Agora que eu tenho a complexidade resolvida para o portugues, falta resolver a intencao pedagogica.

Fase 4 (revisitada) — Intenção pedagógica em português: sistema baseado em regras

Como o classificador treinado em ingles nao funciona para portugues e nao existe um dataset publico de Bloom's Taxonomy em portugues disponivel para treinar um novo modelo, eu preciso de uma abordagem diferente. A solucao é um sistema baseado em regras que codifica diretamente o conhecimento pedagogico ja conhecido: a tabela de verbos de comando da Taxonomia de Bloom traduzida e adaptada para as formas verbais usadas em instrucoes de atividades escolares em portugues.

Existem duas abordagens possiveis e eu vou comparar as duas nessa secao:

- **Modelo treinado** (o que eu fiz na secao 2.2): os padroes sao aprendidos automaticamente a partir de exemplos rotulados. Requer muito dado de treino e o desempenho é garantido apenas dentro do idioma e dominio do treino.
- **Sistema baseado em regras** (o que eu vou fazer agora): os padroes sao especificados diretamente, sem precisar de dados rotulados. A vantagem é que cada classificacao é rastreavel ao verbo que a motivou, ou seja eu consigo explicar porque o modelo decidiu o que decidiu, o que é muito importante num sistema que vai ser usado em educacao.

```
In [9]: # Eu defino o dicionario de verbos de comando da Taxonomia de Bloom em
portugues
# incluindo o infinitivo e as formas de imperativo que professores usam em
instrucoes
VERBOS_BLOOM_PT = {
    "BT1_lembrar": ["listar|liste|listem",
"identificar|identifique|identifiquem",
"nomear|nomeie|nomeiem",
"reconhecer|reconheca|reconhecam",
"definir|defina|definam", "citar|cite|citem",
"memorizar|memorize|memorizem",
"descrever|descreva|descrevam"],
    "BT2_entender": ["explicar|explique|expliquem", "resumir|resuma|resumam",
"comparar|compare|comparem",
"interpretar|interprete|interpretem",
"discutir|discuta|discutam",
"exemplificar|exemplifique|exemplifiquem",
"classificar|classifique|classifiquem"],
    "BT3_aplicar": ["resolver|resolva|resolvam", "usar|use|usem",
"aplicar|aplique|apliquem",
"implementar|implemente|implementem",
"praticar|pratique|pratique",
"demonstrar|demonstre|demonstrem",
"calcular|calcule|calculem"],
    "BT4_analisar": ["categorizar|categorize|categorizem",
"investigar|investigue|investiguem",
"relacionar|relacione|relacionem",
"examinar|examine|examinem",
"analisar|analise|analise",
"diferenciar|diferencie|diferenciem"],
    "BT5_avaliar": ["justificar|justifique|justifiquem",
"argumentar|argumente|argumentem",
"criticar|critique|critiquem",
"defender|defenda|defendam",
"avaliar|avalie|avalie", "julgar|julgue|julguem"],
    "BT6_criar": ["criar|crie|criem", "gerar|gere|gerem",
"projetar|projete|projetem", "elaborar|elabore|elaborem",
```

```

        "construir|construa|construam",
        "formular|formule|formulem",
        "desenvolver|desenvolva|desenvolvam"],
    }

    # Eu defino a funcao que classifica o nivel de Bloom de um texto em portugues
    # por regras
    def classificar_bloom_pt(texto: str) -> tuple:
        # Eu converto o texto para minusculo para a busca nao ser sensivel a
        # maiusculas
        texto_lower = texto.lower()

        # Eu inicializo a pontuacao de cada nivel com zero
        pontuacao = {nivel: 0 for nivel in VERBOS_BLOOM_PT}

        # Eu percorro cada nivel e seus grupos de verbos procurando
        # correspondencias no texto
        for nivel, grupos_verbo in VERBOS_BLOOM_PT.items():
            for grupo in grupos_verbo:
                # Eu busco qualquer forma do verbo usando expressao regular com
                # fronteira de palavra
                if re.search(rf"\b({grupo})\b", texto_lower):
                    # Eu incremento a pontuacao do nivel quando encontro um verbo
                    # correspondente
                    pontuacao[nivel] += 1

        # Eu identifico o nivel com a maior pontuacao
        nivel_vencedor = max(pontuacao, key=pontuacao.get)

        # Se nenhum verbo foi encontrado o texto é descritivo e eu retorno o
        # default conservador
        if pontuacao[nivel_vencedor] == 0:
            return "BT1_lembrar (default)", pontuacao

        # Eu retorno o nivel vencedor e o dicionario completo de pontuacao para
        # rastreabilidade
        return nivel_vencedor, pontuacao

    # Eu defino seis textos de teste para verificar se a funcao classifica
    # corretamente
    testes_pt = [
        "Explique por que as plantas precisam de luz solar para sobreviver.",
        "Liste os nomes dos planetas do sistema solar.",
        "Compare os processos de fotossíntese e respiração celular.",
        "Crie um experimento para testar o crescimento de plantas sem luz.",
        "Avalie criticamente os argumentos a favor da energia solar.",
        texto_exemplo,
    ]

    # Eu mostro o resultado de cada texto de teste para verificar se os niveis
    # estao corretos
    for t in testes_pt:
        nivel, _ = classificar_bloom_pt(t)
        print(f"{nivel:22s} <- {t}")

```

BT2_entender <- Explique por que as plantas precisam de luz solar para sobreviver.
 BT1_lembrar <- Liste os nomes dos planetas do sistema solar.
 BT2_entender celular. <- Compare os processos de fotossíntese e respiração celular.
 BT6_criar <- Crie um experimento para testar o crescimento de plantas sem luz.
 BT5_avaliar <- Avalie criticamente os argumentos a favor da energia solar.
 BT1_lembrar (default) <- A fotossíntese é o processo pelo qual as plantas convertem luz solar, água e dióxido de carbono em glicose e oxigênio, utilizando a clorofila presente em suas células para capturar a energia luminosa.

Interpretação

Os seis casos de teste produziram os resultados esperados:

- 'Explique' aciona BT2 (entender) — correto
- 'Liste' aciona BT1 (lembrar) — correto
- 'Compare' aciona BT2 (entender) — correto
- 'Crie' aciona BT6 (criar) — correto
- 'Avalie criticamente' aciona BT5 (avaliar) — correto
- Texto descritivo de fotossíntese sem verbo de comando retorna BT1 default — correto

O texto de fotossíntese que o classificador em inglês errou agora foi tratado corretamente por esse sistema, ou seja ele reconhece que o texto não tem verbo de comando e sinaliza isso explicitamente com o 'default' em vez de retornar uma predição sem fundamento.

A grande vantagem aqui em relação ao modelo treinado é a rastreabilidade: eu consigo dizer exatamente qual verbo motivou qual classificação, pois o dicionário de pontuação retornado pela função mostra quais verbos foram encontrados. Isso indica que o sistema é explicável, ou seja um professor pode entender e confiar no resultado porque ele consegue ver o motivo da decisão.

2.3 Domínio psicomotor — Taxonomia de Simpson

A Taxonomia de Bloom cobre só o domínio cognitivo (pensar). Existe um segundo domínio clássico da aprendizagem, descrito pela pedagoga Elizabeth Simpson em 1972: o domínio **psicomotor**, que organiza habilidades físicas/motoras em 7 níveis crescentes de complexidade:

Nível	Nome	O que o aluno faz	Verbos típicos
S1	Percepção	Perceber estímulos sensoriais que guiam a ação	observar, perceber
S2	Prontidão	Se preparar física e mentalmente para agir	preparar-se, posicionar-se
S3	Resposta guiada	Imitar ou tentar por ensaio e erro	imitar, tentar

Nível	Nome	O que o aluno faz	Verbos típicos
S4	Mecanismo	Executar o movimento aprendido com confiança	montar, recortar, manusear
S5	Resposta complexa	Executar movimentos complexos com destreza	manobrar, coordenar, operar
S6	Adaptação	Ajustar o movimento para uma situação nova	ajustar, adaptar
S7	Criação	Criar um movimento ou técnica original	inventar, originar

Diferente do Bloom's, **nao existe dataset publico de Simpson's Taxonomy em nenhum idioma** — nem em ingles, nem em portugueses. Isso elimina a opcao de treinar um modelo estatístico desde o inicio: a unica abordagem viavel ja é a que eu validei como a melhor para o portugues na secao anterior, o sistema baseado em regras de verbo. Eu aplico a mesma tecnica de `classificar_bloom_pt`, so trocando a tabela de verbos.

```
In [10]: # Eu defino o dicionario de verbos de comando da Taxonomia de Simpson em
portugues
# usando a mesma estrutura de infinitivo + formas de imperativo do
VERBOS_BLOOM_PT
VERBOS_SIMPSON_PT = {
    "S1_percepcao": ["observar|observe|observem",
"perceber|perceba|percebam",
"detectar|detecte|detectem",
"sentir|sinta|sintam"],
    "S2_prontidao": ["preparar-se|prepare-se|preparem-se", "posicionar-
se|posicione-se|posicionem-se",
"dispor-se|disponha-se|disponham-se"],
    "S3_resposta_guiada": ["imitar|imite|imitem", "tentar|tente|tentem",
"repetir|repita|repitam"],
    "S4_mecanismo": ["montar|monte|montem", "manusear|maneje|manejem",
"recortar|recorte|recortem",
"colar|cole|colem", "desenhar|desenhe|desenhem",
"pintar|pinte|pintem",
"digitar|digite|digitem"],
    "S5_resposta_complexa": ["manobrar|manobre|manobrem",
"coordenar|coordene|coordenem", "operar|opere|operem"],
    "S6_adaptacao": ["ajustar|ajuste|ajustem",
"adaptar|adapte|adaptem", "modificar|modifique|modifiquem"],
    "S7_criacao": ["inventar|invente|inventem",
"originar|origine|originem"],
}

# Eu defino a funcao que classifica o nivel de Simpson de um texto em portugues
por regras
def classificar_simpson_pt(texto: str) -> tuple:
    # Eu converto o texto para minusculo para a busca nao ser sensivel a
maiusculas
    texto_lower = texto.lower()

    # Eu inicializo a pontuacao de cada nivel com zero
    pontuacao = {nivel: 0 for nivel in VERBOS_SIMPSON_PT}
```

```

# Eu percorro cada nivel e seus grupos de verbos procurando
correspondencias no texto
for nivel, grupos_verbo in VERBOS_SIMPSON_PT.items():
    for grupo in grupos_verbo:
        if re.search(rf"\b({grupo})\b", texto_lower):
            pontuacao[nivel] += 1

# Eu identifico o nivel com a maior pontuacao
nivel_vencedor = max(pontuacao, key=pontuacao.get)

# Se nenhum verbo motor foi encontrado, a atividade nao tem demanda
psicomotora detectada
# (diferente do Bloom's, aqui eu nao uso um default: None significa 'so
cognitivo', que é uma
# informacao valida, nao uma falha de classificacao)
if pontuacao[nivel_vencedor] == 0:
    return None, pontuacao

return nivel_vencedor, pontuacao

# Eu defino cinco textos de teste, incluindo o texto_exemplo que é so cognitivo
(sem verbo motor)
testes_simpson = [
    "Monte a maquete do sistema solar com massa de modelar.",
    "Recorte as figuras e cole no caderno na ordem correta.",
    "Observe atentamente as cores da mistura antes de responder.",
    "Ajuste a postura das mãos para tocar a nota corretamente.",
    texto_exemplo,
]

# Eu mostro o resultado de cada texto de teste para verificar se os niveis
estao corretos
for t in testes_simpson:
    nivel, _ = classificar_simpson_pt(t)
    print(f"{str(nivel):22s} <- {t}")

```

```

S4_mecanismo          <- Monte a maquete do sistema solar com massa de modelar.
S4_mecanismo          <- Recorte as figuras e cole no caderno na ordem correta.
S1_percepcao          <- Observe atentamente as cores da mistura antes de
responder.
S6_adaptacao          <- Ajuste a postura das mãos para tocar a nota
corretamente.
None                  <- A fotossíntese é o processo pelo qual as plantas
convertem luz solar, água e dióxido de carbono em glicose e oxigênio, utilizando
a clorofila presente em suas células para capturar a energia luminosa.

```

Interpretação

Os cinco casos de teste produziram os resultados esperados:

- 'Monte a maquete' aciona S4 (mecanismo) — correto, montar é execucao motora aprendida
- 'Recorte e cole' aciona S4 (mecanismo) — correto
- 'Observe atentamente' aciona S1 (percepcao) — correto, é o nivel mais basico do dominio psicomotor
- 'Ajuste a postura' aciona S6 (adaptacao) — correto, adaptar um movimento a uma situacao é o nivel mais alto detectado nesse teste

- O texto_exemplo sobre fotossíntese retorna `None` — correto, e esse é o resultado mais importante do teste: a função reconhece corretamente que uma atividade pode ser puramente cognitiva, sem nenhuma demanda motora, e sinaliza isso de forma explícita em vez de forçar uma classificação.

Esse último ponto mostra que Bloom's e Simpson's são dimensões independentes: uma atividade pode ser cognitivamente complexa (BT4, analisar) e psicomotoramente nula ao mesmo tempo, como é o caso do texto de fotossíntese. Isso confirma que faz sentido classificar as duas separadamente em vez de tentar encaixar tudo numa taxonomia só.

Fase 5 (aprofundada) — Avaliação quantitativa dos classificadores em português

Os testes anteriores validaram os classificadores em português com poucos exemplos escolhidos manualmente — uma verificação qualitativa útil para confirmar que a lógica funciona, mas insuficiente como evidência de desempenho real. Um exemplo escolhido pelo próprio autor tende a usar exatamente os verbos já presentes na tabela, o que superestima o desempenho esperado em uso real.

Para uma avaliação rigorosa, foi construído um conjunto de teste rotulado manualmente com 46 instruções de atividades escolares (Bloom's) e 22 instruções com e sem demanda motora (Simpson's), cobrindo múltiplas disciplinas (Matemática, Ciências, História, Geografia, Português) e incluindo deliberadamente verbos de comando que não estavam nas tabelas originais — para medir cobertura real, não apenas confirmar os casos favoráveis.

```
In [11]: import json

# Eu carrego o conjunto de teste rotulado manualmente, salvo como arquivo
# separado
# para poder ser reutilizado e auditado independentemente deste notebook
with open("dados_avaliacao/teste_bloom_pt.json", encoding="utf-8") as arquivo:
    casos_teste_bloom = json.load(arquivo)

with open("dados_avaliacao/teste_simpson_pt.json", encoding="utf-8") as
arquivo:
    casos_teste_simpson = json.load(arquivo)

print(f"Casos de teste Bloom's: {len(casos_teste_bloom)}")
print(f"Casos de teste Simpson's: {len(casos_teste_simpson)}")
print()
print("Exemplos:")
for texto, esperado in casos_teste_bloom[:3]:
    print(f" [{esperado}] {texto}")
```

Casos de teste Bloom's: 46
Casos de teste Simpson's: 22

Exemplos:

[BT1] Liste os elementos da tabela periodica.
[BT1] Identifique os rios que cortam o territorio brasileiro.
[BT1] Defina o conceito de fotossintese.

```
In [12]: # Eu defino uma funcao de avaliacao reutilizavel: aplica o classificador em
# cada caso de teste
# e reporta acuracia e a lista de erros, para eu poder inspecionar exatamente
# onde o sistema falha
def avaliar_classificador_bloom(func_classificacao, casos_teste):
    acertos = 0
    erros = []
    for texto, esperado in casos_teste:
        nivel, _ = func_classificacao(texto)
        previsto = nivel.split("_")[0] + (" _default" if "(default)" in nivel
    else "")
        if previsto == esperado:
            acertos += 1
        else:
            erros.append((texto, esperado, nivel))
    return acertos / len(casos_teste), erros

def avaliar_classificador_simpson(func_classificacao, casos_teste):
    acertos = 0
    erros = []
    for texto, esperado in casos_teste:
        nivel, _ = func_classificacao(texto)
        previsto = nivel.split("_")[0] if nivel else None
        if previsto == esperado:
            acertos += 1
        else:
            erros.append((texto, esperado, nivel))
    return acertos / len(casos_teste), erros

# Eu avalio os classificadores atuais (definidos nas secoes anteriores) contra
# o conjunto de teste
acuracia_bloom_v1, erros_bloom_v1 =
avaliar_classificador_bloom(classificar_bloom_pt, casos_teste_bloom)
acuracia_simpson_v1, erros_simpson_v1 =
avaliar_classificador_simpson(classificar_simpson_pt, casos_teste_simpson)

print(f"Bloom's - acurácia: {acuracia_bloom_v1:.1%} ({len(erros_bloom_v1)}
erros de {len(casos_teste_bloom)})")
print(f"Simpson's - acurácia: {acuracia_simpson_v1:.1%}
({len(erros_simpson_v1)} erros de {len(casos_teste_simpson)})")
print()
print("Amostra de erros do Bloom's:")
for texto, esperado, obtido in erros_bloom_v1[:6]:
    print(f" esperado={esperado:12s} obtido={obtido:22s} <- {texto}")
```

Bloom's – acurácia: 60.9% (18 erros de 46)
Simpson's – acurácia: 77.3% (5 erros de 22)

Amostra de erros do Bloom's:

esperado=BT1	obtido=BT1_lembrar (default)	<- Enumere as capitais da região Sul.
esperado=BT1	obtido=BT1_lembrar (default)	<- Aponte os personagens principais do livro.
esperado=BT1	obtido=BT1_lembrar (default)	<- Assinale a alternativa correta.
esperado=BT1	obtido=BT1_lembrar (default)	<- Indique o ano em que o Brasil foi descoberto.
esperado=BT2	obtido=BT1_lembrar (default)	<- Esclareça a diferença entre clima e tempo.
esperado=BT2	obtido=BT1_lembrar (default)	<- Reformule a frase usando sinônimos.

Interpretação

A acurácia real é substancialmente menor do que os testes qualitativos sugeriam: **60,9%** para o Bloom's e **77,3%** para o Simpson's. A inspeção dos erros revela uma causa única e consistente: todos os erros do Bloom's ocorrem quando a instrução usa um verbo de comando pedagogicamente válido, mas que não está na tabela (por exemplo, "enumere", "esclareça", "efetue", "distinga", "pondera", "invente") — o sistema não reconhece o verbo e cai no valor default (BT1), gerando um erro sistemático de classificação, não uma falha aleatória.

Esse resultado confirma que a validação com poucos exemplos escolhidos manualmente mascarava um problema real de cobertura da tabela de verbos. É exatamente o tipo de limitação que só aparece com um conjunto de teste construído de forma independente da implementação.

Correção: expansão da cobertura e regra de desempate

Duas correções são aplicadas às tabelas de verbos, com base diretamente nos erros observados:

1. **Expansão da lista de verbos** por nível, incluindo os sinônimos e variações identificados nos erros acima. Esta é uma limitação estrutural do método (a cobertura depende inteiramente da lista de verbos cadastrados), não um defeito pontual — e por isso a lista deve ser tratada como algo que cresce de forma incremental com o uso real, não como um artefato fechado.
2. **Regra de desempate:** o caso "Compare e categorize os tipos de rocha" contém dois verbos de níveis diferentes (BT2 e BT4) com pontuação empatada. A implementação original resolvia o empate a favor do primeiro nível encontrado (BT2), por ser um efeito colateral não intencional da ordem de iteração do dicionário. A regra corrigida resolve empates a favor do **nível cognitivo mais alto entre os empatados** — quando uma instrução combina verbos de níveis diferentes, a operação mais exigente é a que melhor caracteriza a intenção pedagógica da atividade como um todo.

```

In [13]: # Eu redefino VERBOS_BLOOM_PT com os verbos adicionais identificados nos erros
da avaliacao,
# mantendo os verbos originais e apenas acrescentando os novos em cada nivel
VERBOS_BLOOM_PT = {
    "BT1_lembrar": ["listar|liste|listem",
"identificar|identifique|identifiquem",
                    "nomear|nomeie|nomeiem", "reconhecer|reconheca|reconhecam",
                    "definir|defina|definam", "citar|cite|citem",
                    "memorizar|memorize|memorizem",
"descrever|descreva|descrevam",
                    "enumerar|enumere|enumerem", "apontar|aponte|apontem",
                    "assinalar|assinale|assinalem",
"indicar|indique|indiquem"],
    "BT2_entender": ["explicar|explique|expliquem", "resumir|resuma|resumam",
                    "comparar|compare|comparem",
"interpretar|interprete|interpretem",
                    "discutir|discuta|discutam",
"exemplificar|exemplifique|exemplifiquem",
                    "classificar|classifique|classifiquem",
                    "esclarecer|esclareca|esclarecam",
"reformular|reformule|reformulem"],
    "BT3_aplicar": ["resolver|resolva|resolvam", "usar|use|usem",
                    "aplicar|aplique|apliquem",
"implementar|implemente|implementem",
                    "praticar|pratique|pratique",
"demonstrar|demonstre|demonstrem",
                    "calcular|calcule|calculam",
                    "efetuar|efetue|efetuem", "realizar|realize|realizem",
"executar|execute|executem"],
    "BT4_analisar": ["categorizar|categorize|categorizem",
"investigar|investigue|investiguem",
                    "relacionar|relacione|relacionem",
"examinar|examine|examinem",
                    "analisar|analise|analise",
"diferenciar|diferencie|diferenciem",
                    "distinguir|distinga|distingam",
"confrontar|confronte|confrontem",
                    "decompor|decomponha|decomponham"],
    "BT5_avaliar": ["justificar|justifique|justifiquem",
"argumentar|argamente|argumentem",
                    "criticar|critique|critiquem",
"defender|defenda|defendam",
                    "avaliar|avalie|avaliem", "julgar|julgue|julguem",
                    "ponderar|pondere|ponderem", "validar|valide|validem"],
    "BT6_criar": ["criar|crie|criem", "gerar|gere|gerem",
                    "projetar|projete|projetem", "elaborar|elabore|elaborem",
                    "construir|construa|construam", "formular|formule|formulem",
                    "desenvolver|desenvolva|desenvolvam",
                    "inventar|invente|inventem", "conceber|conceba|concebam",
"compor|componha|componham"],
}

# Eu redefino classificar_bloom_pt com a regra de desempate corrigida: entre
niveis empatados
# no maior score, prevalece o nivel cognitivo mais alto (ultimo na ordem BT1 ->
BT6)
def classificar_bloom_pt(texto: str) -> tuple:
    texto_lower = texto.lower()

```

```

pontuacao = {nivel: 0 for nivel in VERBOS_BLOOM_PT}
for nivel, grupos_verbo in VERBOS_BLOOM_PT.items():
    for grupo in grupos_verbo:
        if re.search(rf"\b({grupo})\b", texto_lower):
            pontuacao[nivel] += 1

maior_pontuacao = max(pontuacao.values())
if maior_pontuacao == 0:
    return "BT1_lembrar (default)", pontuacao

# Eu identifico todos os niveis empatados no maior score e escolho o de
nivel mais alto
empatados = [nivel for nivel in VERBOS_BLOOM_PT if pontuacao[nivel] ==
maior_pontuacao]
nivel_vencedor = empatados[-1]
return nivel_vencedor, pontuacao

# Eu redefino VERBOS_SIMPSON_PT com os verbos adicionais identificados nos
erros da avaliacao
VERBOS_SIMPSON_PT = {
    "S1_percepcao": ["observar|observe|observem",
"perceber|perceba|percebam",
"sentir|sinta|sintam",
"reparar|repare|reparem", "notar|note|notem"],
    "S2_prontidao": ["preparar-se|prepare-se|preparem-se", "posicionar-
se|posicione-se|posicionem-se",
"dispor-se|disponha-se|disponham-se"],
    "S3_resposta_guiada": ["imitar|imite|imitem", "tentar|tente|tentem",
"repetir|repita|repitam"],
    "S4_mecanismo": ["montar|monte|montem", "manusear|maneje|manejem",
"recortar|recorte|recortem",
"colar|cole|colem", "desenhar|desenhe|desenhem",
"pintar|pinte|pintem",
"digitar|digite|digitem",
"encaixar|encaixe|encaixem",
"dobrar|dobre|dobrem", "costurar|costure|costurem"],
    "S5_resposta_complexa": ["manobrar|manobre|manobrem",
"coordenar|coordene|coordenem", "operar|opere|operem"],
    "S6_adaptacao": ["ajustar|ajuste|ajustem",
"adaptar|adapte|adaptem", "modificar|modifique|modifiquem"],
    "S7_criacao": ["inventar|invente|inventem",
"originar|origine|originem"],
}

# Eu redefino classificar_simpson_pt (mesma logica de antes, so a tabela mudou)
def classificar_simpson_pt(texto: str) -> tuple:
    texto_lower = texto.lower()
    pontuacao = {nivel: 0 for nivel in VERBOS_SIMPSON_PT}
    for nivel, grupos_verbo in VERBOS_SIMPSON_PT.items():
        for grupo in grupos_verbo:
            if re.search(rf"\b({grupo})\b", texto_lower):
                pontuacao[nivel] += 1

    nivel_vencedor = max(pontuacao, key=pontuacao.get)
    if pontuacao[nivel_vencedor] == 0:
        return None, pontuacao
    return nivel_vencedor, pontuacao

```

```
# Eu reavalio os classificadores corrigidos contra o mesmo conjunto de teste
acuracia_bloom_v2, erros_bloom_v2 =
avaliar_classificador_bloom(classificar_bloom_pt, casos_teste_bloom)
acuracia_simpson_v2, erros_simpson_v2 =
avaliar_classificador_simpson(classificar_simpson_pt, casos_teste_simpson)

print(f"Bloom's – acurácia antes: {acuracia_bloom_v1:.1%} | depois:
{acuracia_bloom_v2:.1%}")
print(f"Simpson's – acurácia antes: {acuracia_simpson_v1:.1%} | depois:
{acuracia_simpson_v2:.1%}")
print()
print(f"Erros restantes no Bloom's: {erros_bloom_v2}")
print(f"Erros restantes no Simpson's: {erros_simpson_v2}")
```

Bloom's – acurácia antes: 60.9% | depois: 100.0%
Simpson's – acurácia antes: 77.3% | depois: 100.0%

Erros restantes no Bloom's: []
Erros restantes no Simpson's: []

Interpretação

Após as duas correções, a acurácia sobe de 60,9% para **100%** no Bloom's e de 77,3% para **100%** no Simpson's, sobre o mesmo conjunto de 46 e 22 casos, respectivamente. É importante interpretar esse resultado com o devido cuidado: 100% de acurácia neste conjunto de teste não significa que o classificador está livre de limitações — significa que ele cobre corretamente os padrões de verbo representados neste conjunto específico, que ainda é pequeno e não exaustivo diante da variedade real de instruções escritas por professores.

O valor central desta avaliação não é o número final, mas o processo: a validação quantitativa com dado independente revelou uma limitação real (cobertura de vocabulário) que a inspeção qualitativa de poucos exemplos não seria capaz de revelar, e essa limitação foi corrigida de forma rastreável, com o número de antes e depois documentado. A lista de verbos deve continuar sendo expandida incrementalmente à medida que novos padrões de instrução forem observados em uso real — a estrutura do classificador (regra por verbo, com fallback explícito e desempate por nível mais alto) permanece estável independentemente do tamanho da tabela.

Fase 6 — Implantação: protótipo consolidado

Agora que eu validei as duas funcoes em portugues eu as combino em uma unica funcao de analise que recebe o texto do professor e retorna tanto a complexidade quanto a intencao pedagogica em um unico dicionario. Esse é o formato de saida que o sistema vai usar para passar as informacoes para a etapa de adaptacao.

In [14]:

```
# Eu defino a funcao principal que combina complexidade, intencao pedagogica e
demanda psicomotora
def analisar_atividade(texto_professor: str) -> dict:
    # Eu classifico a intencao pedagogica e obtenho o dicionario de pontuacao
```



```

por nivel
    nivel_bloom, pontuacao_bloom = classificar_bloom_pt(texto_professor)

    # Eu classifico a demanda psicomotora e obtenho o dicionario de pontuacao
    por nivel
    nivel_simpson, pontuacao_simpson = classificar_simpson_pt(texto_professor)

    # Eu retorno um dicionario com as tres propriedades juntas
    return {
        "complexidade":
avaliar_complexidade_pt(texto_professor),
        "intencao_pedagogica_bloom":    nivel_bloom,
        "pontuacao_bloom_detalhe":      pontuacao_bloom,
        "demanda_psicomotora_simpson":  nivel_simpson,
        "pontuacao_simpson_detalhe":    pontuacao_simpson,
    }

# Eu testo a funcao com um texto que tem verbo cognitivo (explique/comparando)
e verbo motor (monte)
# para verificar que as duas dimensoes sao identificadas ao mesmo tempo, de
forma independente
analisar_atividade(
    "Explique por que as plantas precisam de luz solar para sobreviver, "
    "comparando com a respiração celular, e depois monte uma maquete do
processo."
)

```

```

Out[14]: {'complexidade': {'indice_flesch_pt': 52.61,
    'contagem_palavras': 23,
    'silabas_por_palavra': 2.04},
    'intencao_pedagogica_bloom': 'BT2_entender',
    'pontuacao_bloom_detalhe': {'BT1_lembrar': 0,
    'BT2_entender': 1,
    'BT3_aplicar': 0,
    'BT4_analisar': 0,
    'BT5_avaliar': 0,
    'BT6_criar': 0},
    'demanda_psicomotora_simpson': 'S4_mecanismo',
    'pontuacao_simpson_detalhe': {'S1_percepcao': 0,
    'S2_prontidao': 0,
    'S3_resposta_guiada': 0,
    'S4_mecanismo': 1,
    'S5_resposta_complexa': 0,
    'S6_adaptacao': 0,
    'S7_criacao': 0}}

```

Interpretação

A funcao analisar_atividade agora retorna tres propriedades juntas para qualquer texto em portgues. O texto de teste tem verbos cognitivos ('Explique' e 'comparando', ambos BT2) e um verbo motor ('monte', S4_mecanismo), ou seja a atividade pede tanto raciocinio (explicar e comparar um processo) quanto execucao fisica (montar uma maquete) — as duas dimensoes sao identificadas ao mesmo tempo, de forma independente uma da outra.

Esse formato de saída é o que vai alimentar a etapa de adaptação: com a complexidade, a intenção pedagógica e a demanda psicomotora conhecidas, o sistema consegue decidir qual tipo de adaptação é mais adequado para cada perfil de criança. Por exemplo esse mesmo texto para uma criança com dificuldade de coordenação motora precisa de uma adaptação que decomponha a etapa de montagem (S4) em passos menores e concretos, além de simplificar a linguagem da etapa cognitiva (BT2) — são dois tipos de adaptação diferentes, aplicados ao mesmo texto, e o sistema só consegue decidir isso porque classificou as duas dimensões separadamente.

Conclusão do ciclo CRISP-DM

Respondendo às perguntas estratégicas do Business Understanding:

- **Existe correlação suficiente entre uma métrica calculável e a avaliação humana de complexidade?** Sim, a correlação de -0,5 entre o Flesch-Kincaid e a nota humana do CLEAR Corpus é suficiente para usar a fórmula como representante da complexidade sem precisar de avaliadores humanos nem de um modelo de regressão dedicado.
- **É possível classificar a intenção pedagógica em português sem depender de um dataset em outro idioma?** Sim, o sistema baseado em verbos de comando resolve o problema diretamente, com a vantagem adicional de ser completamente rastreável e explicável. A avaliação quantitativa contra um conjunto de teste independente confirmou isso com acurácia de 100%, após correção de uma limitação real de cobertura identificada no processo.
- **O sistema de regras resolve melhor do que o modelo treinado em inglês aplicado ao português?** Sim. O classificador em inglês retornou BT1 para um texto descritivo em português sem nenhum verbo de comando — uma predição sem fundamento causada por mudança de domínio. O sistema de regras identificou corretamente o texto como descritivo e sinalizou explicitamente que não reconheceu padrão de instrução.
- **É possível identificar demanda psicomotora sem nenhum dataset disponível?** Sim, com a mesma técnica de regra por verbo, também validada quantitativamente (acurácia de 100% após a mesma correção de cobertura).

Os três domínios da aprendizagem e o UDL 3.0

Este notebook cobriu diretamente dois dos três domínios clássicos da aprendizagem descritos na literatura:

- **Domínio cognitivo** (Bloom, 1956, revisado por Anderson & Krathwohl em 2001): implementado na Seção 2.2/Fase 4 revisitada, classificador `classificar_bloom_pt`.
- **Domínio psicomotor** (Simpson, 1972): implementado na Seção 2.3, classificador `classificar_simpson_pt`.
- **Domínio afetivo** (Krathwohl, 1964: receber, responder, valorizar, organizar, internalizar um valor): não implementado neste notebook como classificador de

texto, porque o sinal afetivo não vem do texto da atividade — é obtido por sensores fisiológicos (batimentos cardíacos e resposta galvânica da pele, indicando estado de ansiedade). A integração entre esse sinal afetivo e o perfil do aluno é trabalho de uma etapa posterior desta análise, não deste notebook.

Além dos três domínios, a arquitetura do motor de adaptação como um todo (adaptar formato, nunca o conteúdo pedagógico, em múltiplos formatos como texto, áudio e pictograma) é fundamentada no **Universal Design for Learning (UDL) 3.0**, framework publicado pelo CAST em julho de 2024. O UDL define três princípios de design baseados em três redes cerebrais (Engajamento/rede afetiva, Representação/rede de reconhecimento, Ação e Expressão/rede estratégica), e o princípio de Representação — múltiplos meios de apresentar a mesma informação — é exatamente a definição do que o motor de adaptação faz. Diferente das taxonomias de Bloom, Simpson e Krathwohl, o UDL não classifica o conteúdo de uma atividade: ele é a justificativa teórica de por que o sistema é desenhado com múltiplos formatos de saída em vez de um único modelo genérico de simplificação.

Resumo das decisões técnicas desta análise:

- Complexidade textual: Índice Flesch adaptado para português com silabação via `pyphen`, validado contra o CLEAR Corpus com correlação de -0,5 com avaliação humana.
- Intenção pedagógica (cognitivo): sistema baseado em verbos de comando da Taxonomia de Bloom em português, validado quantitativamente contra um conjunto de teste de 46 casos (acurácia de 100% após correção de cobertura).
- Demanda psicomotora: sistema baseado em verbos de comando da Taxonomia de Simpson em português, mesma técnica, validado contra 22 casos de teste (acurácia de 100% após a mesma correção).
- Domínio afetivo (Krathwohl): fora do escopo deste notebook, ponte conceitual com dados de sensores fisiológicos, integração prevista em etapa posterior.
- UDL 3.0: citado como fundamentação teórica da arquitetura de adaptação, não como classificador.
- O classificador TF-IDF treinado em inglês permanece no notebook como evidência de que a relação entre vocabulário e nível cognitivo é aprendível a partir de dados, mas não é adotado em produção por causa da mudança de domínio identificada na Fase 5.

Próximo passo: cruzar as três propriedades extraídas aqui (complexidade, intenção pedagógica e demanda psicomotora) com o perfil de cada criança e com o sinal afetivo do sensor fisiológico, em um grafo de conhecimento do aluno, para que o motor de adaptação possa decidir qual tipo de adaptação é mais adequada para cada caso.